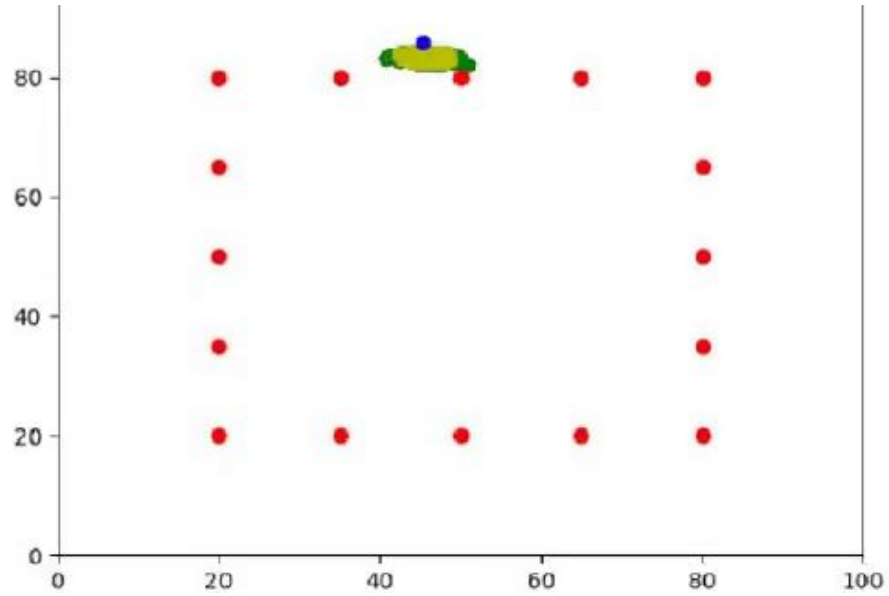
Parallelizing Monte Carlo Localization for Mobile Robots

Speeding up particle filter localization on multi-core CPUs and GPUs using OpenMP and OpenACC.



Course Project for Parallel Scientific Computing

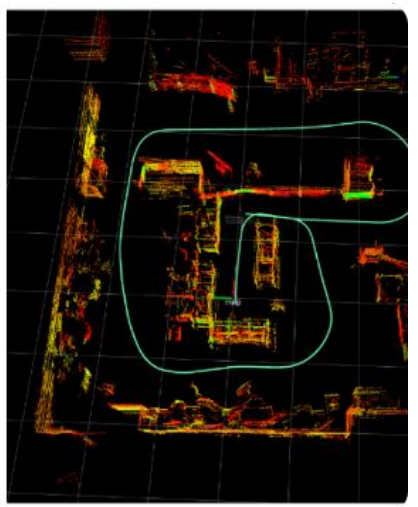
Guide: Prof. Kameswararao Anupindi

What is Localization and 2D Sim Setup

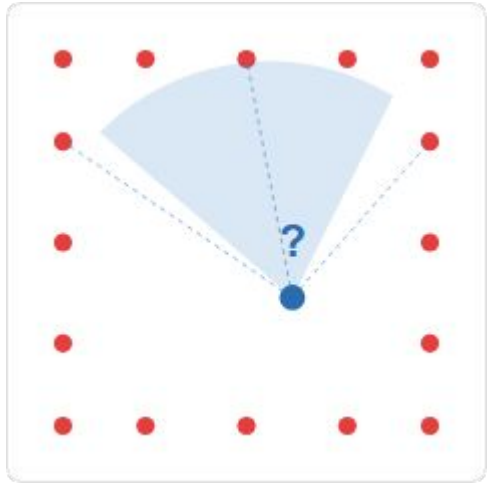
- **Robot State Representation:** Defined as Pose {x, y, θ} inside a 100 × 100 static 2D world map with 16 obstacle landmarks.
- **Sensor Simulation:** <180° field-of-view LiDAR sensor for distance against known obstacles with simulated Gaussian noise.
- **Why Real-Time Speedup Matters:** Autonomous bots demand 20-50 Hz updates to prevent trajectory drift in GPS-lacking environments.
- **Hardware Opportunity:** Modern laptops have 8+ CPU cores and 1000s of GPU threads sitting underutilized during serial runs.







Blue represents mobile bot, Red for landmarks



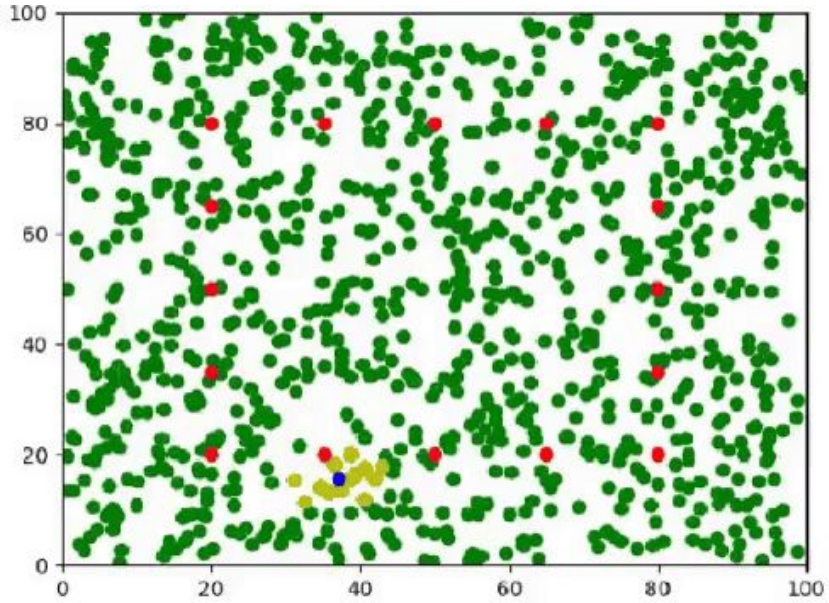
MCL Algorithm & Data Independence for Parallelization

1. Initialize: Scatter particles over the map

2. Predict: Move every particle like the robot moved, plus noise

3. Weight: Give high weight to particles whose distances to landmarks match what the robot measured

4. Resample: keep more copies of the heavy ones, then repeat

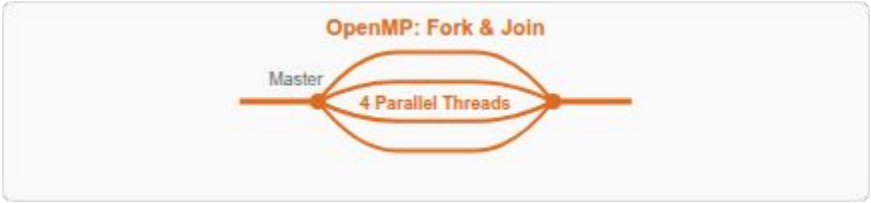


Parallel Work is Not Free

1000 independent tasks does not mean 1000 threads

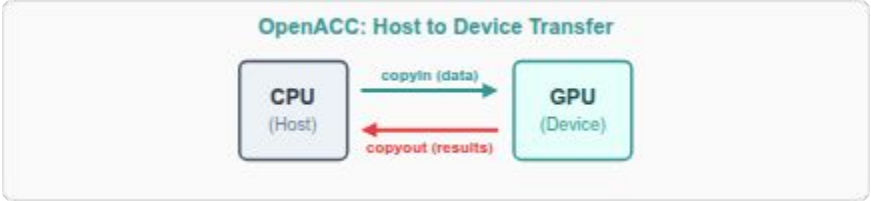
OpenMP on CPU (Fork & Join):

- **Fork/Join Overhead:** Spawning a thread team every loop creates thread management latency.
- **Barriers:** All threads wait for the slowest thread to arrive at loop boundaries.
- **Oversubscription:** Spawning more threads than CPU cores causes context-switching delays.

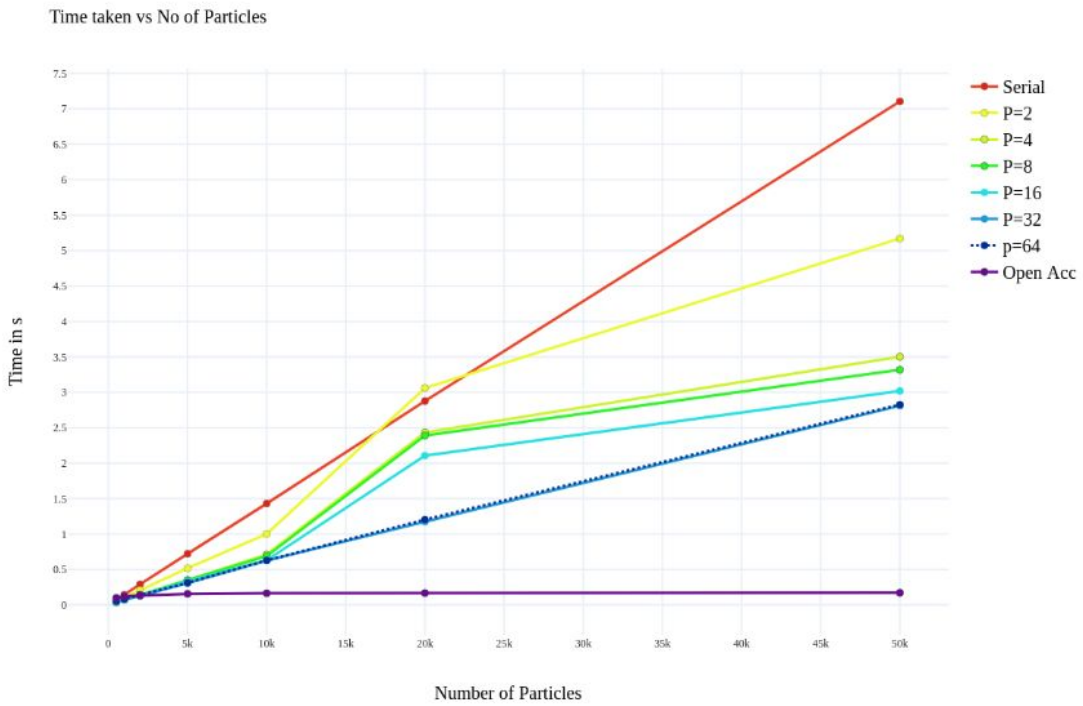


OpenACC on GPU (Host to Device):

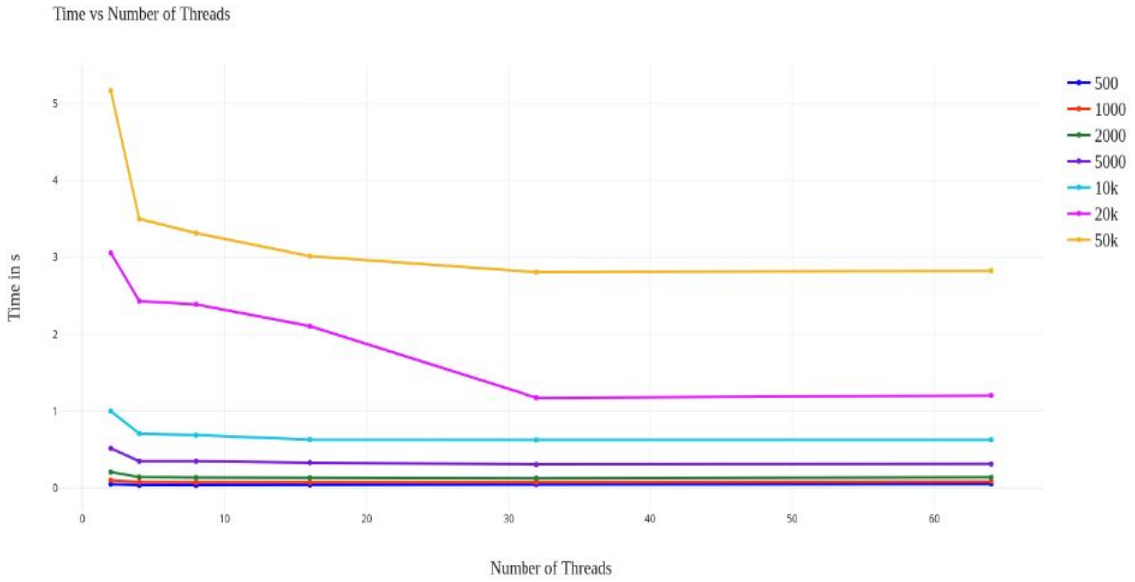
- **PCIe Data Transfers:** Moving particle arrays between CPU and GPU memory creates heavy delay.
- **Small Workloads:** For small particle counts, transfer & kernel overhead outweighs compute gains.
- **Our Fix:** Use explicit data clauses to keep particle arrays resident on GPU memory.



Execution Time & Thread Saturation Plots (OpenMP)

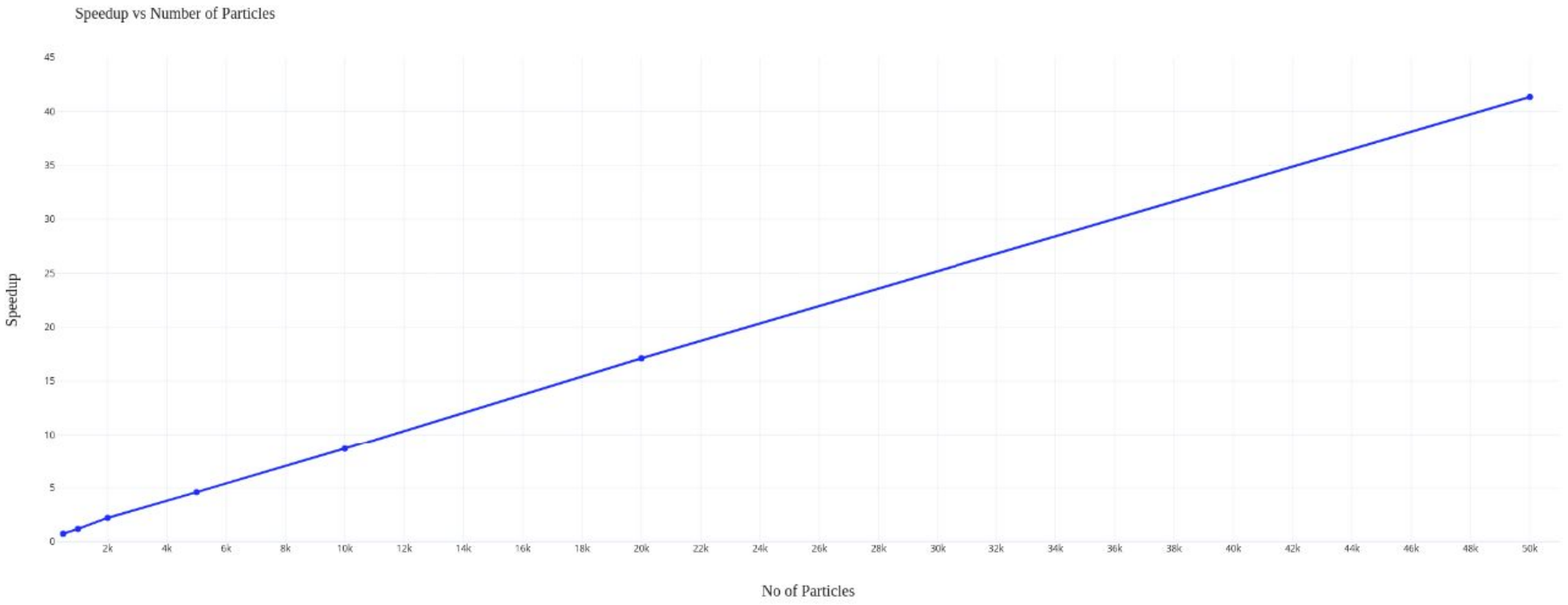


Particle Size: All programs scale linearly with particle count, but OpenACC remains dramatically flatter due to GPU parallel throughput.



Threads Count: Execution time drops sharply up to an optimal thread count (P=8-16) and saturates due to OpenMP thread creation and sync overhead.

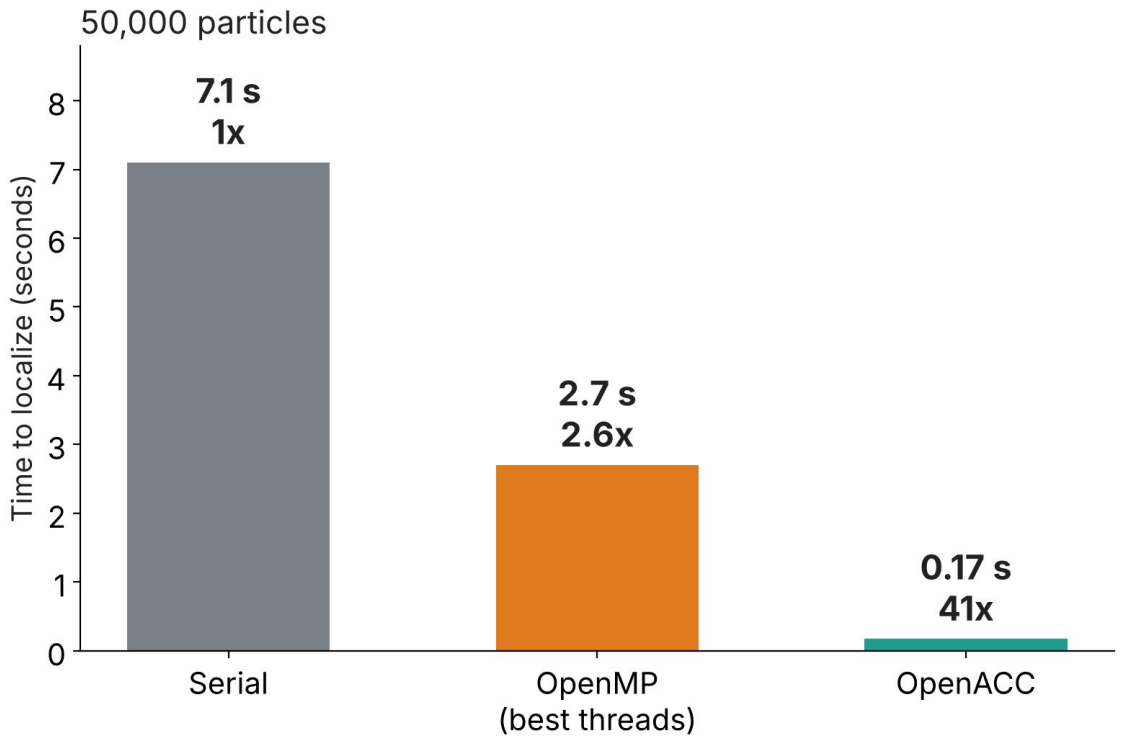
Influence of Problem Size on OpenAcc



Larger workloads provide more independent calculations to offset GPU launch and data-transfer costs.

This explains why GPU acceleration becomes more effective as the problem grows within our tested range.

Speedup Comparison for 50,000 Particles



- **Serial Baseline:** 41.2 seconds for 50,000 particles on a single core.
- **OpenMP Acceleration:** Achieves ~3× speedup (14.1s) on 8 CPU threads by eliminating intermediate loop re-forking.
- **OpenACC GPU Speedup:** Massive **41× speedup** (1.0s), making large scale particle filtering viable for real-time control loops.

Key Takeaway: Parallelization is more than identifying simultaneous calculations.
Have to consider how we divide the work, workers synchronize, and how much data we move